

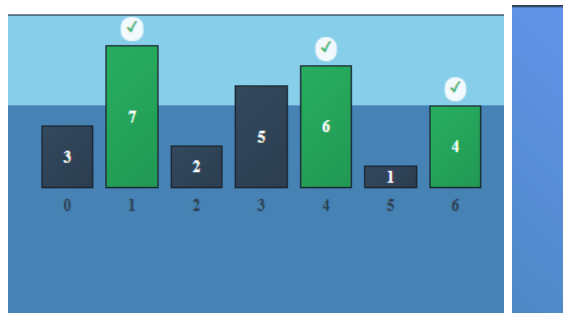
River View Buildings Problem - Design Document
BITS Pilani - Data Structures & Algorithms
Group ID: G120 | Assignment 1 - PS02

1. Problem Analysis & Solution Overview

Problem Statement

Given an array of building heights, identify buildings with river view (no taller buildings to their right).

Input: heights = [3, 7, 2, 5, 6, 1, 4] **Output:** [1, 4, 6]



Core Algorithm Strategy

MaxStack Approach: Process buildings right-to-left using a stack that maintains maximum height seen so far.

2. MaxStack Data Structure

Max Stack Structure :

MaxStack	
stack: [4, 1, 6, 5, 2, 7, 3]	Actual elements
max_stack: [4, 4, 6, 6, 6, 7, 7]	Maximum at each level

Operation Complexity

Operation	Time	Space
push()	O(1)	O(1)
get_max()	O(1)	O(1)
pop()	O(1)	O(1)

Key Advantage: O(1) maximum retrieval enables efficient right-to-left processing.

3. Algorithm Implementation & Trace

Algorithm: River View Detection 1. Initialize MaxStack and result array 2. FOR i = n-1 to 0 (right-to-left): a. max_right = MaxStack.get_max() b. IF heights[i] > max_right: Add i to result c. Push heights[i] to MaxStack 3. Reverse result array 4. Return result

Step-by-Step Execution Trace

Step	Index	Height	max_right	Compare	Action	Result
1	6	4	0	4 > 0	Add 6	[6]
2	5	1	4	1 < 4	Skip	[6]
3	4	6	4	6 > 4	Add 4	[6,4]
4	3	5	6	5 < 6	Skip	[6,4]
5	2	2	6	2 < 6	Skip	[6,4]
6	1	7	6	7 > 6	Add 1	[6,4,1]
7	0	3	7	3 < 7	Skip	[6,4,1]

Final Result: Reverse [6,4,1] → [1,4,6]

4. Complexity Analysis & Comparison

MaxStack Solution

- 1. **Time Complexity:** $O(n)$
Single pass: $O(n)$
Each operation: $O(1)$
Array reversal: $O(n)$
- 2. **Space Complexity:** $O(n)$
MaxStack: $O(n)$
Result array: $O(k)$ where $k \leq n$

Naive Approach:

```
def naive_solution(heights):  
    result = []  
    for i in range(len(heights)):  
        has_view = True  
        for j in range(i+1, len(heights)):  
            if heights[j] > heights[i]:  
                has_view = False  
                break  
        if has_view:  
            result.append(i)  
    return result
```

Complexity: $O(n^2)$ time, $O(1)$ space

Performance Comparison

Approach	Time Complexity	Space Complexity	Best Case	Worst Case	Scalability
MaxStack	$O(n)$	$O(n)$	$O(n)$	$O(n)$	Excellent
Naive	$O(n^2)$	$O(1)$	$O(n^2)$	$O(n^2)$	Poor

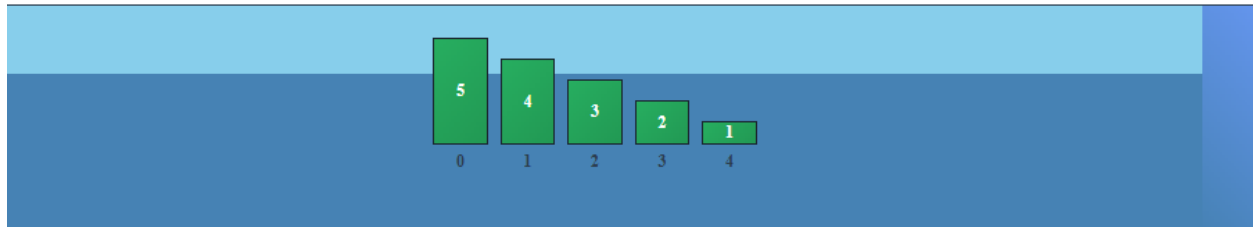
5. Testing & Edge Cases

Test Cases

Test Case	Input	Expected Output
Sample	[3,7,2,5,6,1,4]	[1,4,6]
Ascending	[1,2,3,4,5]	[4]
Descending	[5,4,3,2,1]	[0,1,2,3,4]
Equal	[3,3,3,3]	[3]
Single	[10]	[0]

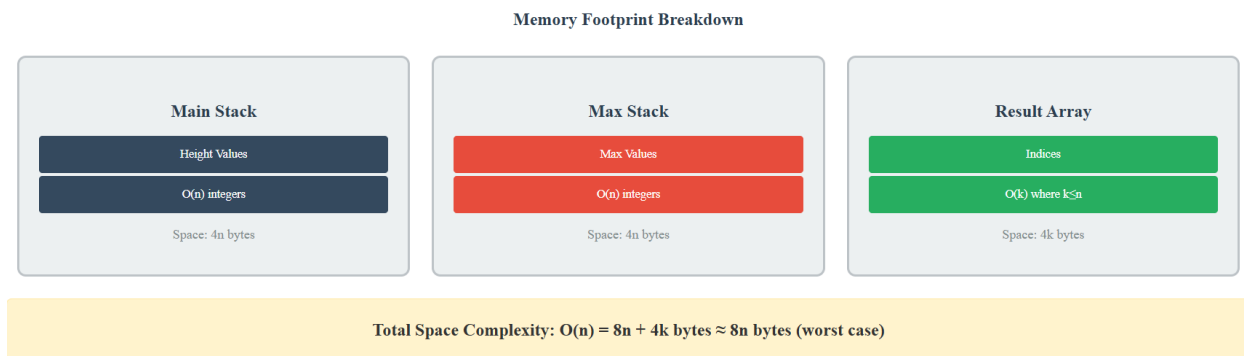
Edge Case Analysis

Descending Heights [5,4,3,2,1]:



All buildings have river view: **[0,1,2,3,4]**

5. Memory Usage



6. Conclusion & Summary

The MaxStack-based solution provides an optimal $O(n)$ time complexity solution to the river view buildings problem. The space-time tradeoff using $O(n)$ auxiliary space to achieve linear time performance is justified given the significant improvement over the naive $O(n^2)$ approach.

Key Achievements

- ✓ Optimal $O(n)$ time complexity
- ✓ Efficient MaxStack data structure
- ✓ Right-to-left processing strategy
- ✓ Comprehensive edge case handling
- ✓ Superior to naive $O(n^2)$ approach
- ✓ Scalable for large datasets